

PROYECTO FINAL - PGII

PENSAMIENTO COMPUTACIONAL

Juan Felipe Ibarra Londoño
Johan Stiven Pineda Martinez
Samuel Arturo Rodriguez Trujillo

1. Abstracción

La abstracción consiste en identificar la esencia del problema, separar información relevante de la irrelevante y modelar el sistema con las entidades mínimas necesarias para representar el dominio de forma coherente y mantenible.

1.1. ¿Qué se solicita finalmente?

El sistema debe proveer una plataforma completa de logística urbana “Same-day” que permita:

- Registro e inicio de sesión de usuarios.
- Gestión de perfil, direcciones y métodos de pago.
- Crear, cotizar, modificar y cancelar envíos.
- Aplicar servicios adicionales (seguro, frágil, prioridad).
- Asignar repartidores y rastrear estados (envío).
- Registrar pagos y generar comprobantes.
- Consultar historial y generar reportes CSV/PDF.
- Panel administrativo para usuarios, repartidores, métricas y asignación operativa.

Todo esto sobre una aplicación JavaFX, con lógica desacoplada mediante servicios.

1.2. ¿Qué información es relevante dado el problema anterior?

La información relevante corresponde estrictamente a los conceptos del dominio logístico:

Entidades principales

- Usuario

- Repartidor
- Envío
- Dirección
- Pago
- Tarifa

Estructuras auxiliares

- Estados de envío (Solicitud, Asignado, En ruta, Entregado.)
- Servicios Adicionales (Seguro, frágil, firma, rastreo...)
- Métricas (Ingresos, tiempos de entrega, incidencias, zonas...)

1.3. ¿Cómo se agrupa la información relevante?

Se agrupa mediante un modelo por capas, siguiendo principios SOLID:

Capa 1 - Dominio (Modelos)

Agrupar las entidades: Usuario, Envío, Repartidor, Dirección, Pago, Tarifa... y demás clases que hacen parte del modelo.

Capa 2 - Lógica de negocio (Servicios)

Servicios independientes con responsabilidad única:

- ILogistica Mapping
- Observador Envío
- Servicio Envío
- Tarifa
- Servicio Autenticación

Capa 3 - Presentación (JavaFX)

Controladores por pantalla + DTOs para desacoplar la lógica del dominio

- Controller
- ViewController
- FXMLs
- DTOs
- Mappers

Capa 4 - Factories y Patrones

Factories para crear entidades y DTOs sin acoplar lógica a constructores

- EmpresaLogisticaFactory
- Patrones (Creacionales, Estructurales, Comportamiento)

1.4. ¿Qué funcionalidades se solicitan finalmente?

Funcionalidades de Usuario

- Registrarse, iniciar sesión
- Gestionar perfil
- Crear, editar y cancelar envíos
- Pagar envíos
- Rastrear estado
- Descargar historial en CSV/PDF

Funcionalidades de Repartidor

- Registrarse, iniciar sesión
- Gestionar Perfil
- Cambiar estado de envío asignado
- Filtrar envíos por estado
- Reportar incidencias
- Visualización de estadísticas de perfil
- Visualizar mapa generado (simulación)

Funcionalidades de Administrador

- Gestión de Usuarios y repartidores
- Gestión de disponibilidades
- Reasignación de envíos
- Gestión de incidencias y cambios
- Gestión de panel de métricas y JavaFX charts

2. Descomposición

La descomposición consiste en dividir un sistema complejo en submodulos manejables, coherentes y fáciles de probar.

2.1. ¿Cómo se distribuyen las funcionalidades?

- Capa UI

Despliega elementos gráficos y dispara eventos de botones con `onAction="nombreDelMetodo"`

- Controlador Login

Responsabilidades:

- Obtener datos de UI
- Validar Login
- Abrir ventana de registro
- Abrir ventana de perfil seleccionado
- Recibir resultados del registro

Métodos clave:

- handleIniciarSesion()
- handleRegistrarse

- **Controlador Registro**

Responsabilidades:

- Capturar campos (user, password, perfil...).
- Validar campos
- Retornar datos al controlador Login

- **Navegación y ventanas**

Responsabilidades:

- Crear Stage
- Cargar FXML
- Conectar controladores
- Interacción con las funcionalidades de cada perfil

2.2. ¿Qué debo hacer para probar las funcionalidades?

La estructura está diseñada para cubrir:

- UI (JavaFx)
- Controladores
- Lógica del Dominio
- Modelos
- Flujo de datos entre pantalla
- Validaciones
- Navegación
- Funcionalidades de negocio

El sistema permite que un usuario se registre o inicie sesión para acceder a sus funcionalidades principales. Una vez dentro, el usuario podrá gestionar su perfil y direcciones frecuentes las cuales se usan como origen y destino de los envíos. Para crear un nuevo envío, el usuario ingresa los datos básicos y el sistema valida la información general que queda registrada en el historial; internamente se activa el funcionamiento de cotizar el valor del envío mediante unos parámetros específicos. El usuario puede consultar el estado de cada envío y realizar recargas de dinero por medio de la cartera digital.

3. Reconocimiento de Patrones

El reconocimiento de patrones consiste en identificar problemas conocidos y aplicar soluciones arquitectónicas probadas.

3.1. ¿Que puedo reutilizar de la solución de otros problemas anteriores?

Patrones Creacionales

- **Singleton**

Propósito: Garantiza una única instancia del factory que gestione todos los datos del sistema.

Beneficio: Control centralizado de datos, evita inconsistencias.

- **Factory Method**

Propósito: Encapsular la creación de objetos complejos.

- **Builder**

Propósito: Construcción paso a paso de objetos complejos.

Patrones Estructurales

- **Decorator**

Propósito: Agregar servicios adicionales al envío (Seguro, Frágil, etc.).

- **Adapter**

Propósito: Adaptar entidades del dominio a DTOs para la capa de presentación.

- **Facade**

Propósito: Simplificar la interfaz factory para los ViewControllers.

Patrones de Comportamiento

- **Strategy**

Propósito: Seleccionar algoritmo de cálculo de tarifa en tiempo de ejecución.

- **Observer**

Propósito: Los cambios de estado del envío notifican cambios en el sistema para todos los perfiles

- **State**

Propósito: Gestionar estados del envío (Solicitado, Confirmado, En Ruta, Entregado, Cancelado).